Loops in Emacs Lisp

This article in the Emacs series explores looping techniques that are available with Emacs Lisp.

There are built-in constructs such as *while* and *dolist* that are shipped with the default GNU Emacs. The dash.el library provides functions to iterate over lists, and is written by Magnar Sveen. The latest release of dash.el is v2.16.0, and its source code is available at *https://github.com/magnars/dash.el* under the GNU General Public License v3.0.

Let us also explore the structures available in loop.el, another library for implementing imperative loops. This has been written by Wilfred Hughes, and has also been released under the GNU General Public License v3.0.

Installation

The dash.el and loop.el packages are available in Milkypostman's Emacs Lisp Package Archive (MELPA) and in the Marmalede repo. You can install the package using the following commands in GNU Emacs:

```
M-x package-install dash
M-x package-install loop
```

The other method of installation is to copy the dash.el and loop.el source files to your Emacs load path and load them. In order to get syntax highlighting of dash functions in Emacs buffers, you can add the following command to your Emacs initialisation settings:

```
(eval-after-load 'dash '(dash-enable-font-lock))
```

If you are using Cask (*https://github.com/cask/cask*) to manage your Emacs configuration, then you can simply add the following code to your Cask file:

```
(depends-on "dash")
(depends-on "loop")
```

The usage of various loop construct is as follows.

Built-in

GNU Emacs has built-in loop constructs. The *while* function, for example, has the following syntax:

```
(while TEST BODY...)
```

The BODY code segment is evaluated if the result of TEST is not nil. Until TEST returns nil, the BODY will continue to be executed. An example of *while* function usage is given below:

```
(setq alphabets '(a b c d e))

(defun print-list-elements (list)
  "Print each element of the input LIST"
  (while list
    (print (car list))
    (setq list (cdr list))))

(print-list-elements alphabets)
```

The output is as follows:

```
a
b
c
d
e
nil
```

The 'dolist' macro loops over a list and is also built-in with Emacs. Its definition is as follows:

```
(dolist (VAR LIST [RESULT]) BODY...)
```

The VAR argument represents each element in LIST for every iteration in the BODY segment. The value in RESULT is returned by the function, and is optional. By default, a *nil* is returned. The 'alphabet' list elements can be printed using the *dolist* macro as shown below:

```
(setq alphabets '(a b c d e))
(dolist (element alphabets)
  (print element))
```

The resultant output is the same.

```
a
b
c
d
e
nil
```